

Morphic Studio

Document 30 — Deployment & Infrastructure (Expanded Specification)

Document ID: MS-030

Version: 2.0

Status: Expanded Specification

Priority: Critical Engineering Infrastructure

Purpose

The Deployment & Infrastructure Architecture defines how Morphic Studio is developed, deployed, hosted, maintained, monitored, and scaled throughout its lifecycle.

It establishes a cloud-first, modular architecture that enables a solo developer to build a professional-grade creative platform while supporting future growth without major architectural changes.

Vision

Development should be:

- Simple enough for a beginner to understand.
- Professional enough to scale.
- Modular enough to evolve.
- Reliable enough for creators to trust.

Infrastructure should disappear into the background, allowing developers to focus on building features rather than managing servers.

Core Philosophy

Build Small. Design Big. Scale Gradually.

Version 1 is intentionally lightweight, but every architectural decision should support future expansion.

Development Environment

Primary Development Machine:

- Lenovo IdeaPad S345
- FydeOS
- Linux environment enabled

Primary Development Platform:

- Replit

Primary UI Design Platform:

- Google Stitch

Primary Version Control:

- Git
- GitHub

Primary AI Development Assistants:

- ChatGPT
- Google Gemini

These tools together provide an efficient cloud-first workflow suitable for the project's early stages.

High-Level Architecture

Client (Browser)

↓

Frontend Application

↓

Backend API

↓

Workflow Engine



AI Services



Storage Layer



Database



Export Pipeline

Each layer has a clear responsibility and communicates through defined interfaces.

Frontend

Responsibilities:

- User interface
- User interactions
- Project editing
- Dashboard
- Asset browsing
- Workflow visualization
- Real-time updates (future)

The frontend should remain independent from AI implementation details.

Backend

Responsibilities:

- Business logic
- Authentication
- Authorization
- Workflow orchestration

- Database access
- AI coordination
- Export management
- Notifications

The backend acts as the central coordinator for the platform.

Database

Primary responsibilities:

- User accounts
- Projects
- Story Bible
- Creative Memory
- Assets
- Workflows
- Permissions
- Metadata
- Activity logs

The database should store structured information rather than large binary files whenever possible.

Storage Layer

Stores:

- Images
- Audio
- Videos
- Exports
- AI assets
- Project backups
- Temporary files

The storage layer should remain separate from the database for scalability.

AI Layer

The AI layer is composed of modular services responsible for:

- Story analysis

- Character generation
- Background generation
- Animation support
- Voice generation
- Audio generation
- Creative planning

Each service communicates through standardized interfaces.

Workflow Engine

The Workflow Engine coordinates:

- AI jobs
- Rendering
- Asset generation
- Notifications
- Exports
- Background processing

It prevents complex production pipelines from becoming tightly coupled.

Deployment Strategy

Development

↓

Testing

↓

Staging (future)

↓

Production

Version 1 may combine Development and Production due to the small user base, but the architecture should support separate environments later.

Source Control

Every change should follow:

Feature Branch

↓

Development

↓

Review

↓

Merge

↓

Deployment

Git history serves as the project's engineering memory.

Configuration Management

Configuration should be externalized using environment variables.

Examples include:

- Database connection
- Authentication secrets
- AI provider credentials
- Storage configuration
- Email service settings

Configuration should never be hardcoded.

Logging

The platform should record:

- Application logs
- Workflow logs
- AI execution logs
- Authentication events
- Error logs
- Deployment logs

Logs support debugging and future monitoring.

Backup Strategy

Backups include:

- Database backups
- Asset backups
- Configuration backups
- Export archives

Recovery procedures should be documented and tested.

Deployment Pipeline

Future deployment pipeline:

Code Commit

↓

Automated Checks

↓

Build

↓

Tests

↓

Deployment

↓

Verification

↓

Monitoring

Automation should increase over time as the platform matures.

Scalability Strategy

Version 1:

- Solo developer
- 3–5 users
- Lightweight infrastructure

Future versions:

- Additional users
- Larger asset libraries
- Multiple AI providers
- Distributed processing
- Dedicated infrastructure

Scaling should occur incrementally rather than through major redesigns.

User Stories

As the developer, I want an environment that is easy to maintain.

As a creator, I want the platform to remain reliable during long creative sessions.

As a future administrator, I want infrastructure that can grow without disrupting users.

Functional Requirements

The infrastructure shall:

- Deploy the application.
 - Manage configuration.
 - Coordinate services.
 - Support backups.
 - Enable updates.
 - Record logs.
 - Recover from failures.
 - Support future scaling.
-

Non-Functional Requirements

The infrastructure should:

- Be maintainable.
 - Be modular.
 - Minimize downtime.
 - Support gradual scaling.
 - Separate concerns.
 - Encourage automation.
-

Error Handling

If deployment fails:

- Preserve the previous working version.
- Record deployment logs.
- Notify administrators.
- Support rollback procedures.

If infrastructure components become unavailable:

- Isolate failures.
 - Preserve user work.
 - Retry operations where appropriate.
-

Future Expansion

Future versions may include:

- Continuous Integration (CI)
- Continuous Deployment (CD)
- Containerization
- Orchestrated services
- Distributed rendering
- GPU worker pools
- Global asset delivery
- Multi-region deployment
- Enterprise infrastructure

These capabilities should build upon the existing architecture rather than replace it.

Acceptance Criteria

The Deployment & Infrastructure Architecture succeeds when Morphe Studio can be developed efficiently by a solo creator today while remaining capable of supporting significantly larger workloads in the future without fundamental architectural changes.

Key Principle

Infrastructure exists to support creativity, not distract from it. By building with modularity, automation, and scalability in mind, Morphe Studio can grow naturally from a personal project into a professional creative platform.

End of Document 30